Week 2 - Revision

Logic synthesis, physical design, verification, test and post-GDS processes

Course Lessons 06-09 plus the Tcl tutorial (Lesson 10). This guide combines your new handwritten points with the remaining Week 2 material. The nine-page revision is followed by four pages of your Week 2 handwriting. Week 1's behavioural synthesis guide remains separate. [1-6]

1. Synthesis | 2. Terms | 3. Planning | 4. Clocks/routes | 5. Verification | 6. Test/yield | 7. After GDS | 8. Tcl | 9. Recall/sources | Your handwriting

1. Logic synthesis: RTL to a cell netlist

Logic synthesis translates synthesizable RTL into an equivalent circuit made from available library cells, while optimizing power, performance and area under constraints. It produces a **netlist**: instances plus their connections. Physical design subsequently determines placement and routing. Fabrication comes later. [2]

Input	What it supplies	What it does not supply
RTL, commonly Verilog/VHDL	Clocked state updates and combinational behaviour	Final cell coordinates or wire geometry
Cell library, commonly Liberty `.lib`	Cell functions, timing, power and area models	The required behaviour of the top design
Constraints, commonly SDC	Clocks, I/O timing, loads and design intent	A guarantee that the requirements are achievable

SDC means Synopsys Design Constraints. Its commands use Tcl syntax in an EDA environment; an SDC command is not automatically a standalone Tcl command. Different RTL and structural netlists may both use the `.v` extension: inspect the contents to identify the abstraction.

Three synthesis levels: behavioural synthesis starts with an algorithm and chooses a timed RTL architecture; RTL synthesis infers generic operators and storage; technology mapping chooses actual cells. Tools often combine several of these operations in one synthesis flow. Your notes' two synthesis diagrams refer to different starting abstractions.

Read the result: a conditional choice feeding an edge-triggered register can become selection logic followed by a flip-flop. The mapper may choose one mux cell or an equivalent gate network. A cell library change, tighter clock requirement or different output load can change that network even when the RTL stays the same.

Check yourself: can a legal netlist still miss timing? Yes: functional equivalence and constraint satisfaction are separate requirements. A synthesis report based on estimated wires is also not final routed timing signoff.

2. Elaboration, optimization and circuit vocabulary

Synthesis sequence: parse and elaborate RTL; infer and simplify generic hardware; map to the target library; optimize the mapped circuit. Elaboration resolves hierarchy, parameters, widths and connectivity. Typical optimization propagates constants, removes unused logic and restructures Boolean expressions while preserving the required behaviour. Mapping and optimization can iterate. [2]

Generic logic describes function without committing to a particular characterized implementation. There is no single universal generic gate set. A generic AND and a chosen standard-cell AND have the same intended Boolean function, but the latter has specific area, drive and timing properties.

Term	Concrete interpretation
Library cell	Reusable type, for example `AN2`
Library pin	Pin definition on that type, for example `AN2/A`
Instance	One use of a type: `AN2 U1` and `AN2 U2` are two instances
Instance pin	`U1/A` and `U2/A` are different connection points
Port	Interface signal on a module/design boundary
Net	Connection among pins and/or ports; it is not itself a gate
Fanout	Number of connected loads, under the stated counting convention

The slash is explanatory hierarchy notation; actual tool naming can differ. Connecting `U1/Y` to `U2/A` requires a net. Two pins both called `A` do not connect merely because they share a local pin name.

Why cell choice matters: a stronger cell can improve a loaded output's transition while adding area, input capacitance or power. Delay depends on the arc, input slew, load and operating conditions; it is not one universal number per gate. **Slew** measures edge transition time; **propagation delay** measures the response between specified input/output crossings.

FSM boundary: a finite-state machine becomes state storage, next-state logic and output logic. Detailed state minimization belongs to a later course week. The unrelated FSM exercises at the beginning of your uploaded file are preserved in its complete source copy; they are not presented as Week 2 requirements.

3. Physical design: chip planning and placement

Physical design turns logical connectivity into manufacturable geometry. A useful introductory sequence is **chip planning -> placement -> CTS -> global routing -> detailed routing -> extraction, checks and ECO iteration**. Actual flows can revisit earlier stages. [3]

Stage	Main decision	Distinction to remember
Partitioning	Divide a large design into interacting blocks	Communication and hierarchy affect the result
Floorplanning	Set die/core size, macro/I/O locations and standard-cell regions/rows	Establishes the large-scale organization
Power planning	Build supply and ground distribution, or PDN	Supports voltage integrity and reliability
Placement	Choose legal locations for individual standard-cell instances	Optimizes estimated wire length, timing and routability

Your chip-planning note about standard-cell locations means allocating regions and rows. Detailed coordinates for individual cells belong to placement. A fixed hard macro cannot simply be reshaped like a flexible block region.

PDN means power delivery network. In a simple DC segment, voltage drop is $\Delta V = IR$. With 20 mA through 0.5 ohm, the drop is 10 mV. A real network has distributed loads and transient effects. More supply wiring can improve delivery while consuming routing resources.

Utilization: state the denominator. If $U = A_{cells}/A_{usable}$, then 0.60 mm^2 of cells at 60% utilization needs 1.00 mm^2 of usable placement area. That is not automatically the whole die area: macros, margins, blockages and reserved regions matter.

Congestion is local routing demand relative to available routing capacity. Dense or poorly arranged placement can create a shortage even at moderate average utilization. Before routing, wire length and delay are estimated; placement can still optimize them. Short estimated wire length alone does not prove that all pins can be routed legally.

4. Clocks, routing, file views and ECO

Clock tree synthesis (CTS) creates a clock distribution network using buffers/inverters and routing structures. It manages sink latency, skew, load and transitions. **Latency** is arrival delay from a chosen clock reference to a sink. **Skew** is the arrival difference between specified sinks. Equal 1 ns arrival delays give zero skew, not zero latency. [3]

With sink arrivals of 10 ps and 15 ps, the arrival difference is 5 ps. Equal wire lengths alone do not ensure equal delays: cell delay, electrical loading and parasitics also matter. A gated clock needs a glitch-safe structure; a changing enable on a naive AND gate can create an unintended clock edge.

Global routing plans routes through regions and allocates coarse routing resources, often by layer. **Detailed routing** selects exact legal wires and vias, respecting spacing, width, pin-access and other rules. A promising global route can still encounter detailed-routing violations.

View/file	Primary purpose
Liberty `.lib`	Functional and characterized timing/power cell data
LEF	Abstract physical dimensions, pins, obstructions; technology routing data
Netlist `.v`	Instances and logical connectivity
SDC	Clock and timing/design intent constraints
DEF / implementation database	Placed/routed design state and related implementation information
SPEF	Modeled/extracted interconnect parasitic resistance and capacitance
GDSII	Detailed hierarchical layout geometry for release/manufacturing preparation

LEF is Library Exchange Format. An abstract pin shape does not replace a timing model, and a GDS layout does not by itself state every timing requirement.

ECO means engineering change order: a controlled implementation change, such as a local buffer, resizing or connectivity correction. It can affect timing, power, area and verification, so relevant checks must be repeated. Extraction supplies parasitics for more realistic timing/electrical analysis. Finishing routing is not the same as finishing signoff.

5. Verification: choose the right evidence

Design verification asks whether the design satisfies its specification. **Manufacturing test** screens fabricated parts for defects and performance requirements. Design-for-test hardware is planned during design; that does not make simulation a physical chip test. [4]

Method/check	What it establishes within its model	Main limitation or distinction
Simulation	Responses for applied stimuli and modeled timing	Passing selected cases is not exhaustive correctness
Model checking	Whether specified properties hold over the modeled behaviours	Depends on properties, assumptions and tractability
Equivalence checking	Agreement between two representations under the defined comparison	Both can agree while implementing a wrong specification
Static timing analysis (STA)	Timing checks over constrained paths and delay models	Needs correct clocks, exceptions, corners and constraints
Design rule check (DRC)	Layout compliance with supplied geometric/process rules	Does not prove intended circuit connectivity or function
Layout versus schematic (LVS)	Extracted layout connectivity/devices match the reference	Does not establish all timing or design-rule requirements
Electrical rule check (ERC)	Specified electrical connection/voltage rules	Rule set and tool determine the checks included

CEC means combinational equivalence checking. Formal methods include model checking and equivalence checking; those are not synonyms. The exact sequential comparison depends on the selected method and assumptions.

STA vocabulary: sequentially adjacent registers are connected by a path containing combinational logic/interconnect without intervening state elements. They need not be physically adjacent. Setup checks late arrival; hold checks early change.

For a simple single-cycle path with zero skew and uncertainty, $T \geq t_{CQ,\,max} + d_{comb,\,max} + t_{setup}$. With 0.08, 0.62 and 0.10 ns respectively, the minimum modeled period is 0.80 ns, or 1.25 GHz. Increasing the period can help setup; it does not generally cure hold. Detailed STA is taught later.

6. Test, yield, fault coverage and escapes

A **defect** is a physical imperfection; a **fault model** represents a possible erroneous circuit behaviour; a **failure** is an observed violation. A tester applies patterns and compares measured and expected responses at defined conditions. **ATE** means automatic test equipment. Wafer probing provides electrical access before packaging. [4]

Activate and observe: to detect an AND-gate input stuck at 0, drive both inputs to 1. The good output becomes 1 and the faulty output 0. Driving the other input to 0 masks the difference. Testability depends on controllability and observability, not just on applying many patterns.

Fault coverage is detected modeled faults divided by the stated modeled-fault population. Specify whether the denominator excludes untestable faults. **Yield** is the fraction of manufactured units meeting the stated goodness criteria. Neither percentage alone determines the defective fraction among shipped passing parts.

Your yield-model note is written as:

$$Y = \left(1 + \frac{Ad}{\alpha}\right)^{-\alpha}, \qquad A, d \geq 0, \quad \alpha > 0.$$

A is the modeled die/critical area, d the matching relevant defect density and α the clustering parameter. Area and density must use reciprocal units so Ad is dimensionless. For fixed d, α, increasing A lowers this model's yield. For $Ad = 1$, $\alpha = 1$ gives 0.5 and $\alpha = 2$ gives about 0.444; the large-α limit is e^{-Ad}, about 0.368. Stronger clustering (smaller α here) can leave more dies untouched at fixed mean defect density. This is a model, not a universal prediction for every process. [7]

Your 100-chip example, with its extra assumption made explicit: 90 chips are good; suppose a test detects half of the 10 bad chips and rejects no good chips. Five bad chips escape, 95 pass, and the defective fraction among passing chips is $5/95 \approx 5.26\%$, or 52,632 ppm. The assumed 50% bad-chip detection probability is not automatically the same as 50% modeled fault coverage. Testing screens parts; without repair it does not turn bad silicon into good silicon.

7. After GDS: masks, fabrication and packaged parts

Release route: layout data preparation -> mask manufacture -> wafer fabrication -> wafer test -> dicing -> assembly/packaging -> final test and classification. Screening and test order can vary by product. GDS release is a design milestone, not a completed packaged chip. [5]

Fracturing converts layout polygons into shapes supported by a mask writer. In the lecture's conventional transmissive-mask example, a pattern is written into resist on a chromium-coated transparent blank, developed, transferred by etching and inspected. Mask writing manufactures the mask; wafer exposure later uses the mask to pattern wafer resist.

Concept	Mechanism and purpose	Cost or limitation
Optical proximity correction (OPC)	Precompensate mask geometry for predicted printing distortion; e.g. line ends/corners	Process-dependent correction and mask complexity
Double/multiple patterning	Decompose a dense target into separate patterning groups/steps	Extra processing and alignment/overlay requirements
FEOL	Front end of line: forms device structures	Must meet device/process requirements
BEOL	Back end of line: builds much of the interconnect stack	Wire/via parasitics and reliability remain significant

OPC and multiple patterning can be used together. A modified mask corner need not be the desired final wafer corner. Your 193 nm note refers to a DUV lithography technology, not to every process or every minimum feature width.

Packaging supplies external connections, mechanical protection and a thermal path; it also adds electrical parasitics. DIP is **dual in-line package**; BGA is **ball grid array**. Wafer test avoids packaging some failing dies; final test catches faults introduced or revealed by later processing.

Burn-in is specified electrical/thermal stress used where appropriate to screen early-life weaknesses. **Binning** classifies parts by measured speed, power, leakage or functional capability. On-chip delay-measurement circuits may help speed classification, but they are not the definition of binning. A lower speed bin can still meet its own specification completely. Binning is not a resolution-enhancement technique.

8. Tcl: read substitutions before predicting output

Tcl = Tool Command Language. A script contains commands; the first word names the command and the other words are its arguments. Whitespace separates words; newline or semicolon separates commands when not grouped. An EDA tool embeds Tcl and adds its own commands. [6,8]

Syntax	Meaning at the ordinary Tcl parsing stage
`set n 3`	Assign 3 to variable `n`; `set n` retrieves it
`$n`	Substitute a variable's value
`[expr {1+2}]`	Run the enclosed command and substitute its result
`"value=$n"`	Group one word while allowing substitutions
`{value=$n}`	Group one word; suppress ordinary substitutions at this stage
`expr {$n + 2}`	Expression evaluator computes arithmetic; braces defer its evaluation

```tcl
set n 3
puts "n=$n"              ;# prints n=3
puts {$n [expr {1+2}]}  ;# prints $n [expr {1+2}] literally
puts [expr {$n+2}]      ;# prints 5
set values {1 2 3}
set total 0
foreach x $values {
    incr total $x
}
puts $total            ;# prints 6
```

Lists: `llength` counts elements, `lindex` reads a zero-based element, `lset` updates an element of a named list variable and `lappend` adds elements. Use `list` to preserve argument boundaries, including paths containing spaces. A Tcl array is a key-indexed variable collection; it is not the same data structure as a list.

Control and files: `if`, `for`, `while` and `foreach` control execution; `proc` defines a procedure and `return` returns a result. `source script.tcl` evaluates a script. `open` returns a channel; `gets`/`read`, `puts` and `close` use it. `exec` runs an external program. `get_cells` or `create_clock` require the appropriate EDA application's commands, not just plain `tclsh`.

Trap: braces do not mean "never evaluated." A command such as `expr`, `if` or `foreach` can evaluate the expression/script it receives later. A comment starts where Tcl expects a new command; the `;#` above deliberately starts a comment after a command.

9. Last-pass recall and source map

Answer these aloud before opening the practice questions:

1. Give the three main synthesis inputs and the netlist output. Distinguish HLS, RTL synthesis and mapping.

2. Identify a library cell, instance, instance pin, port and net in one small circuit.

3. Explain chip planning versus placement, and global versus detailed routing.

4. Distinguish clock latency from skew, and setup from hold.

5. Choose between simulation, model checking, equivalence, STA, DRC, LVS and ERC for a stated problem.

6. Separate defect, fault, failure, yield, modeled fault coverage and escaping bad parts.

7. Explain why clustering assumptions and the shipped-population denominator matter.

8. Trace GDS to a packaged chip; compare OPC, multiple patterning, burn-in and binning.

9. Predict Tcl substitutions with braces, quotes, variables and square brackets.

Sources and deeper explanations

1. Official NPTEL weekly syllabus: Week 2's implementation, verification/test and post-GDS scope.

2. Lesson 06: logic synthesis.

3. Lesson 07: physical design.

4. Lesson 08: verification and test.

5. Lesson 09: layout to chip.

6. Lesson 10: Tcl tutorial.

7. NPTEL: yield models and data analysis: clustering and the Poisson limit.

8. Official Tcl syntax manual.

Full explanations, figures and earlier complete handwriting remain in the Day 01 reading PDF (Lesson 06) and Day 02 reading PDF (Lessons 07-10). Day 02 also starts Week 3; that later Verilog material is outside this guide.

Your new material: the following four pages reproduce the Week 2 portion of your upload (lower part of original page 3 and complete pages 4-6). The complete six-page original retains its cover and earlier FSM exercises. Read the typed clarifications when a shorthand statement is ambiguous; the original handwriting is preserved as written. No assignment answer key is printed here.

Your notes: synthesis

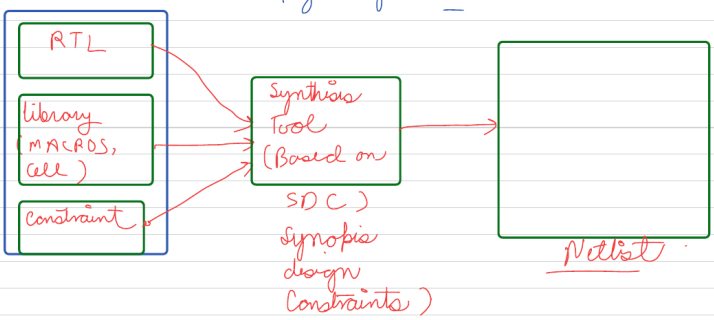
Original page 3, Week 2 portion. The earlier FSM exercise above the Week 2 heading is excluded from this view; the full source is preserved.

Week 2

behavioural Synthesis — ① Diagrams both.

logical synthesis — ② (RTL synthesis (RTL → generic logic gates))

logic Synthesis.



→ Define : pin, Port, instance pins, instance, net

I1/A — instance, Pin A

cell inside a design is instance.

- net wire that connects diff instances & Ports.

Synthesis

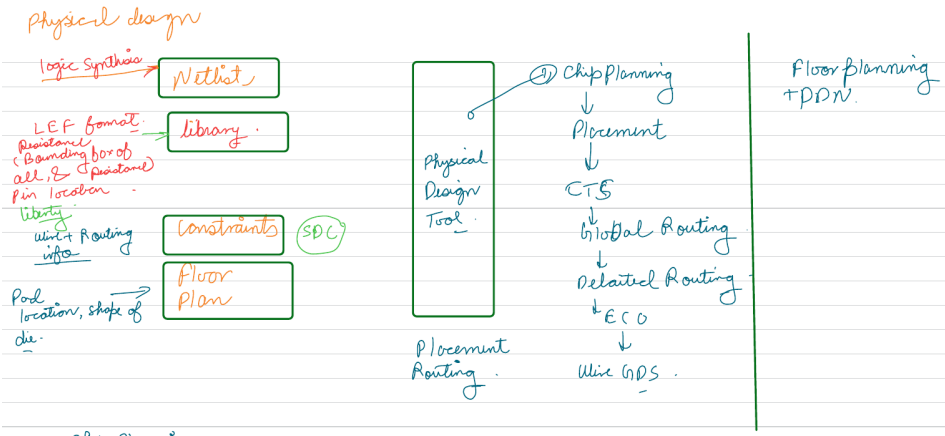
logical synthesis — ② (RTL synthesis (RTL → generic logic gates))

RTL
↓
Netlist (generic logic Gates)
↓
optimized Netlist (logic optimization) (Generic Gate)
↓
Netlist (standard cell)
↓
optimized Netlist)

PPA improve
Area driven.
logic optimization (How if Variables↑)
Technology Mapping (Based on design +Constraints)
We can Make Delay, Power, timing / More Accurate optimization

Your notes: physical design

Complete original page 4. Typed sections 3-5 expand chip planning, placement, CTS, routing and verification, and clarify their boundaries.



• Chip Planning

- Partitioning into subsystems
- location of standard cells
- Allocation of area for the standard cells, macros, memory.
- Power Planning (PDN)

(2) Placement.
→ location of standard cells.
— highly automated
— Wire length minimization (not have done Routing yet)
→ Timing is met ; Delay in critical path ↓
→

Diff B/w chip Planning & Placement

(3) CTS.
Clock skew.

(4) Routing → Global → Divide into Routing Regions.
Local — Allocating wires on each layer.

(5) ECO

(6)

Verification

→ Simulation
(not exhaustive)

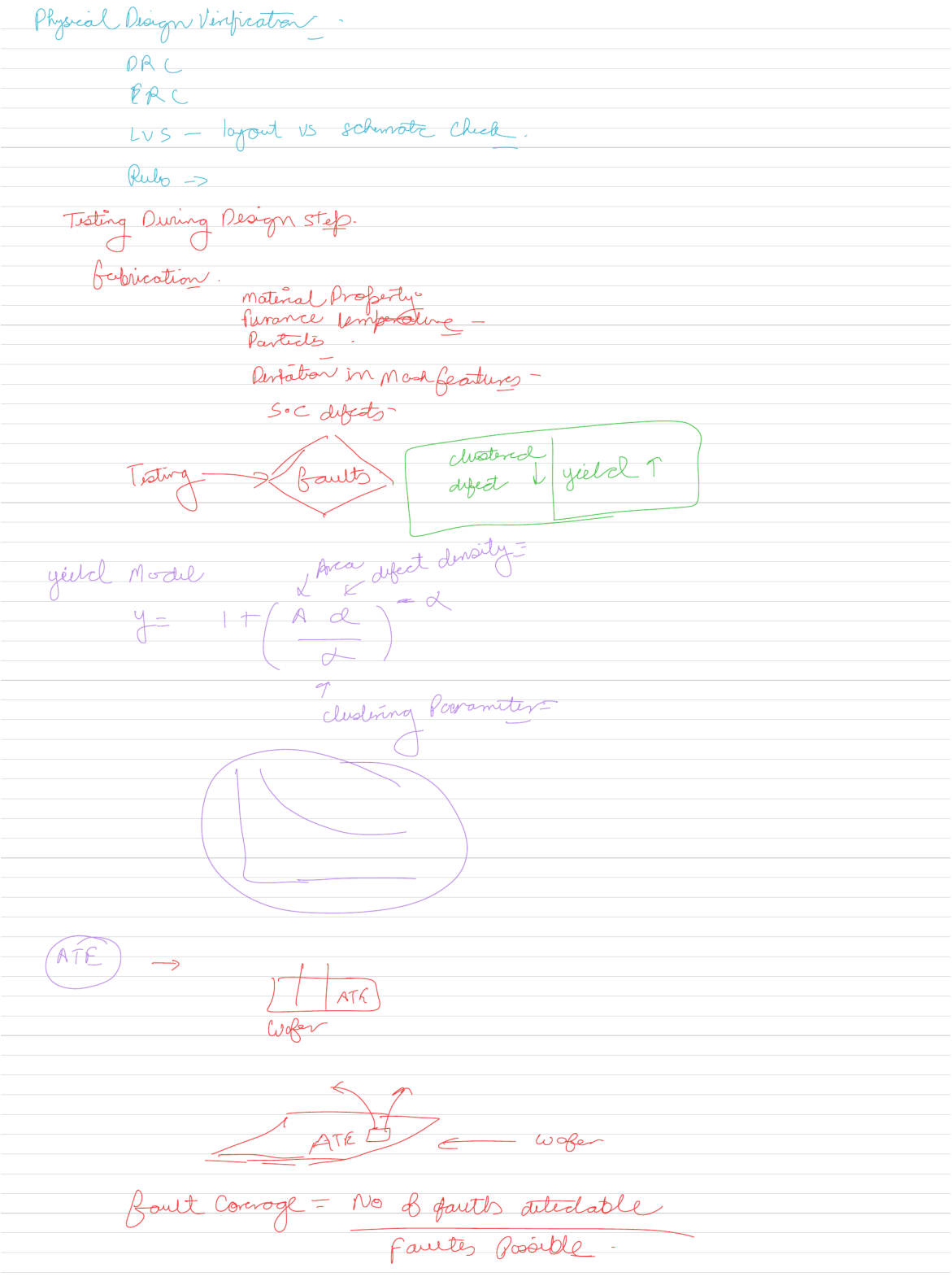
— Model Checking
formal
methods

CEC - combinational equivalence Checking
└ functional equivalent using formal methods

STA → Statically Adjacent flip flop

Your notes: checks, yield and ATE

Complete original page 5. Typed sections 5-6 distinguish verification from manufacturing test and explain the yield model and coverage denominator.



Your notes: escapes and manufacturing

Complete original page 6. Typed sections 6-7 make the 100-chip example's assumption explicit and expand OPC, patterning, FEOL/BEOL, packaging and binning.

